rather than what they can do. Even the `IsInRole` method you have been using checks for a specific claim.

The general advice is to favor claims over roles. Claims offer much more flexibility than roles, which were primarily included in Identity for backward compatibility reasons. Having said that, there is at least one area where roles can still provide value: as a mechanism for bulk-assigning claims to users.

We looked at the schema of the Identity tables last chapter. One of the created tables we didn't explore at the time is the `AspNetRoleClaims` table. This table holds claims and their values that have been assigned to a role rather than an individual user. Any user placed in the specified role automatically acquires all the related claims.

The `RoleManager` class has an `AddClaimsAsync` method, which takes an `IdentityRole` object and a `Claim` as arguments and adds them to this table. Using the existing ClaimsManager/Assign page as a basis, create an additional page called AssignToRole that uses this method to add claims to a role. The main differences between the original Assign page and this new one is that the new one will take a `RoleManager <IdentityRoles>` as a dependency instead of a `UserManager<CityBreaksUser>`. You will use the `RoleManager.Roles` property to get a list of roles for the select list and the `FindByIdAsync` method to hydrate a role from the selected identity value. The rest is up to you, dear reader. If you get stuck, check the downloads for this chapter for a working version.

10.4 Authorizing resources